

TOPIC-7

1.Hamiltonian Path — P/NP Verification

AIM

To implement a program to determine whether a given graph contains a Hamiltonian Path and demonstrate the verification of a proposed Hamiltonian Path.

ALGORITHM

1. Read the vertices and edges of the graph.
2. Generate/check a candidate path containing every vertex exactly once.
3. Verify that:
 - Every vertex occurs exactly once.
 - Every consecutive pair of vertices has an edge between them.
4. If all conditions are satisfied, print that a Hamiltonian Path exists.
5. For the given graph, $A \rightarrow B \rightarrow C \rightarrow D$ is a valid Hamiltonian Path.

SOURCE CODE

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define N 4
5
6 int graph[N][N] = {
7     {0, 1, 0, 1},
8     {1, 0, 1, 0},
9     {0, 1, 0, 1},
10    {1, 0, 1, 0}
11 };
12 bool verifyHamiltonianPath(int path[]) {
13     bool visited[N] = {false};
14
15     for (int i = 0; i < N; i++) {
16         if (path[i] < 0 || path[i] >= N)
17             return false;
18
19         if (visited[path[i]])
20             return false;
21
22         visited[path[i]] = true;
23     }
24
25     bool verifyHamiltonianPath(int path[]) {
26         for (int i = 0; i < N; i++) {
27             if (visited[path[i]])
28                 return false;
29
30             visited[path[i]] = true;
31             if (i > 0 && graph[path[i - 1]][path[i]] == 0)
32                 return false;
33         }
34
35         return true;
36     }
37 }
38
39 int main() {
40     int path[N] = {0, 1, 2, 3};
41
42     if (verifyHamiltonianPath(path)) {
43         printf("Hamiltonian Path Exists: True\n");
44         printf("Path: A -> B -> C -> D\n");
45     } else {
46         printf("Hamiltonian Path Exists: False\n");
47     }
48
49     return 0;
50 }
```

SAMPLE INPUT AND OUTPUT

Output

```
Hamiltonian Path Exists: True
Path: A -> B -> C -> D
```

```
=== Code Execution Successful ===
```

TIME COMPLEXITY

For verification of a given path:

$O(V)$

For finding a Hamiltonian Path by exhaustive backtracking:

$O(V!)$

SPACE COMPLEXITY

$O(V)$

RESULT

Thus, the candidate path $A \rightarrow B \rightarrow C \rightarrow D$ is successfully verified as a Hamiltonian Path. Hamiltonian Path is an NP problem because a proposed path can be verified in polynomial time.

2. 3-SAT and NP-Completeness Verification

AIM

To implement a solution for the 3-SAT problem and demonstrate its NP-completeness by considering a reduction from the NP-complete Vertex Cover problem.

ALGORITHM

1. Represent the 3-SAT formula as a collection of clauses.
2. Generate possible truth assignments for all variables.
3. Evaluate every clause for each assignment.
4. If at least one assignment makes every clause true, the formula is satisfiable.
5. A reduction from Vertex Cover to 3-SAT can be used to establish NP-hardness.
6. Since 3-SAT is also in NP, it is NP-complete.

SOURCE CODE

```
1 #include <stdio.h>
2 #include <stdbool.h>
3 #define VARS 5
4 #define CLAUSES 3
5 typedef struct {
6     int a, b, c;
7 } Clause;
8 Clause formula[CLAUSES] = {
9     {1, 2, -3},
10    {-1, 2, 4},
11    {3, -4, 5}
12 };
13 bool literalValue(int literal, int assignment[]) {
14     int var = literal > 0 ? literal : -literal;
15     if (literal > 0)
16         return assignment[var];
17     else
18         return !assignment[var];
19 }
20 bool clauseSatisfied(Clause clause, int assignment[]) {
21     return literalValue(clause.a, assignment) ||
22            literalValue(clause.b, assignment) ||
23            literalValue(clause.c, assignment);
24 }
25
26 int main() {
27     bool found = false;
28     for (int mask = 0; mask < (1 << VARS); mask++) {
29         for (int i = 1; i <= VARS; i++)
30             assignment[i] = (mask >> (i - 1)) & 1;
31         if (formulaSatisfied(assignment)) {
32             found = true;
33             printf("Satisfiability: True\n");
34             printf("Example satisfying assignment:\n");
35             for (int i = 1; i <= VARS; i++)
36                 printf("%d = %s\n", i, assignment[i] ? "True" : "False");
37         }
38     }
39     if (!found)
40         printf("Satisfiability: False\n");
41     printf("NP-Completeness Verification:\n");
42     printf("Reduction from Vertex Cover to 3-SAT is conceptually successful.\n");
43     return 0;
44 }
```

SAMPLE INPUT AND OUTPUT

```
Output

Satisfiability: True
Example satisfying assignment:
x1 = False
x2 = False
x3 = False
x4 = False
x5 = False
NP-Completeness Verification:
Reduction from Vertex Cover to 3-SAT is conceptually successful.

=== Code Execution Successful ===
```

TIME COMPLEXITY

There are 2^n possible assignments.

$O(2^n \times m)$

where n = number of variables and m = number of clauses.

SPACE COMPLEXITY

$O(n + m)$

RESULT

Thus, the given 3-SAT formula is satisfiable. 3-SAT belongs to NP, and because Vertex Cover can be polynomially reduced to 3-SAT, 3-SAT is NP-complete.

3. Approximation Algorithm for Vertex Cover

AIM

To implement a 2-approximation algorithm for Vertex Cover and compare its result with the exact solution obtained using brute force.

ALGORITHM

Approximation

1. Start with an empty vertex cover.
2. Select an uncovered edge (u,v).
3. Add both u and v to the vertex cover.
4. Remove/mark all edges incident to u or v as covered.
5. Repeat until all edges are covered.

Exact Solution

1. Enumerate all subsets of vertices.
2. Check whether each subset covers every edge.
3. Select the smallest valid vertex cover.
4. Compare the approximation size with the optimal size.

SOURCE CODE

```
1 #include <stdio.h>
2 #include <stdbool.h>
3 #define V 5
4 #define E 5
5 typedef struct {
6     int u, v;
7 } Edge;
8 Edge edges[E] = {
9     {1, 2},
10    {1, 3},
11    {2, 3},
12    {3, 4},
13    {4, 5}
14 };
15 bool contains(int cover[], int size, int vertex) {
16     for (int i = 0; i < size; i++)
17         if (cover[i] == vertex)
18             return true;
19     return false;
20 }
21 bool isCover(int mask) {
22     for (int i = 0; i < E; i++) {
```

```
21 bool isCover(int mask) {
22     for (int i = 0; i < E; i++) {
23         int v = edges[i].v;
24         if (!(mask & (1 << (u - 1))) &&
25             !(mask & (1 << (v - 1))))
26             return false;
27     }
28     return true;
29 }
30
31 int main() {
32     bool covered[E] = {false};
33     int approx[V], approxSize = 0;
34     for (int i = 0; i < E; i++) {
35         if (!covered[i]) {
36             int u = edges[i].u;
37             int v = edges[i].v;
38             approx[approxSize++] = u;
39             approx[approxSize++] = v;
40             for (int j = 0; j < E; j++) {
41                 if (edges[j].u == u || edges[j].v == u ||
42                     edges[j].u == v || edges[j].v == v)
43                     covered[j] = true;
44             }
45         }
46     }
47 }
```

```

31 int main() {
32     for (int i = 0; i < E; i++) {
33         if (!covered[i]) {
34             // ...
35         }
36     }
37     printf("Approximation Vertex Cover: {");
38     for (int i = 0; i < approxSize; i++) {
39         printf("%d", approx[i]);
40         if (i != approxSize - 1)
41             printf(", ");
42     }
43     printf("\n");
44     int bestMask = 0;
45     int bestSize = V + 1;
46     for (int mask = 0; mask < (1 << V); mask++) {
47         int size = 0;
48         for (int i = 0; i < V; i++)
49             if (mask & (1 << i))
50                 size++;
51         if (size < bestSize && isCover(mask)) {
52             bestSize = size;
53         }
54     }
55     printf("Exact Vertex Cover (Brute-Force): {");
56     for (int i = 0; i < bestSize; i++) {
57         printf("%d", bestMask & (1 << i));
58         if (i != bestSize - 1)
59             printf(", ");
60     }
61     printf("\n");
62     printf("Approximation Size = %d\n", approxSize);
63     printf("Optimal Size = %d\n", bestSize);
64     printf("Performance Ratio = %.2f\n",
65           (double)approxSize / bestSize);
66     return 0;
67 }

```

```

31 int main() {
32     for (int mask = 0; mask < (1 << V); mask++) {
33         if (size < bestSize && isCover(mask)) {
34             bestMask = mask;
35             bestSize = size;
36         }
37     }
38     printf("Exact Vertex Cover (Brute-Force): {");
39     for (int i = 0; i < bestSize; i++) {
40         printf("%d", bestMask & (1 << i));
41         if (i != bestSize - 1)
42             printf(", ");
43     }
44     printf("\n");
45     printf("Approximation Size = %d\n", approxSize);
46     printf("Optimal Size = %d\n", bestSize);
47     printf("Performance Ratio = %.2f\n",
48           (double)approxSize / bestSize);
49     return 0;
50 }

```

SAMPLE INPUT AND OUTPUT

```

Output

Approximation Vertex Cover: {1, 2, 3, 4}
Exact Vertex Cover (Brute-Force): {1, 2, 4}
Approximation Size = 4
Optimal Size = 3
Performance Ratio = 1.33

=== Code Execution Successful ===

```

TIME COMPLEXITY

Approximation:

$O(E)$

Brute-force exact algorithm:

$O(2^V \times E)$

SPACE COMPLEXITY

$O(V + E)$

RESULT

The approximation algorithm produces a vertex cover in polynomial time, while brute force finds the exact minimum cover. The approximation algorithm has a guaranteed 2-approximation bound.

4. Greedy Approximation for Set Cover

AIM

To implement the greedy approximation algorithm for the Set Cover problem and compare its result with the optimal solution obtained using exhaustive search.

ALGORITHM

1. Start with an empty collection of sets.
2. Find the set covering the largest number of currently uncovered elements.
3. Add that set to the solution.
4. Mark its elements as covered.
5. Repeat until all elements in the universe are covered.
6. Use exhaustive search to find the minimum number of sets for comparison.

SOURCE CODE

```
1 #include <stdio.h>
2 #include <stdbool.h>
3 #define U 7
4 #define S 6
5 int sets[S][U] = {
6     {1, 1, 1, 0, 0, 0, 0}, // {1,2,3}
7     {0, 1, 0, 1, 0, 0, 0}, // {2,4}
8     {0, 0, 1, 1, 1, 1, 0}, // {3,4,5,6}
9     {0, 0, 0, 1, 1, 0, 0}, // {4,5}
10    {0, 0, 0, 0, 1, 1, 1}, // {5,6,7}
11    {0, 0, 0, 0, 0, 1, 1} // {6,7}
12 };
13 void printSet(int index) {
14     printf("{");
15     int first = 1;
16     for (int j = 0; j < U; j++) {
17         if (sets[index][j]) {
18             if (!first)
19                 printf(", ");
20             printf("%d", j + 1);
21             first = 0;
22         }
23     }
24 }
```

```
13 void printSet(int index) {
24     printf("{");
25 }
26 bool coversAll(int mask) {
27     bool covered[U] = {false};
28     for (int i = 0; i < S; i++) {
29         if (mask & (1 << i)) {
30             for (int j = 0; j < U; j++)
31                 if (sets[i][j])
32                     covered[j] = true;
33         }
34     }
35     for (int i = 0; i < U; i++)
36         if (!covered[i])
37             return false;
38     return true;
39 }
40 int main() {
41     bool covered[U] = {false};
42     int selected[S];
43     int count = 0;
44 }
```

```
41 int main() {
42     int selected[S];
43     int count = 0;
44     while (1) {
45         int bestSet = -1;
46         int bestGain = 0;
47         for (int i = 0; i < S; i++) {
48             int gain = 0;
49             for (int j = 0; j < U; j++)
50                 if (sets[i][j] && !covered[j])
51                     gain++;
52             if (gain > bestGain) {
53                 bestGain = gain;
54                 bestSet = i;
55             }
56         }
57         if (bestSet == -1)
58             break;
59         selected[count++] = bestSet;
60         for (int j = 0; j < U; j++)
61             if (sets[bestSet][j])
62                 covered[j] = true;
63     }
64 }
```

```
41 int main() {
80     for (int mask = 0; mask < (1 << S); mask++) {
81         int size = 0;
82         for (int i = 0; i < S; i++)
83             if (mask & (1 << i))
84                 size++;
85         if (size < bestCount && coversAll(mask)) {
86             bestCount = size;
87             bestMask = mask;
88         }
89     }
90     printf("Optimal Set Cover: ");
91     int first = 1;
92     for (int i = 0; i < S; i++) {
93         if (bestMask & (1 << i)) {
94             if (!first)
95                 printf(", ");
96             printSet(i);
97             first = 0;
98         }
99     }
100     printf("\n");
101     printf("Greedy uses %d sets.\n", count);
102     printf("Optimal uses %d sets.\n", bestCount);
103     return 0;
104 }
```

SAMPLE INPUT AND OUTPUT

Output

```
Greedy Set Cover: {3, 4, 5, 6}, {1, 2, 3}, {5, 6, 7}
Optimal Set Cover: {1, 2, 3}, {2, 4}, {5, 6, 7}
Greedy uses 3 sets.
Optimal uses 3 sets.
```

```
=== Code Execution Successful ===
```

TIME COMPLEXITY

Greedy algorithm:

$O(S^2 \times U)$ in this straightforward implementation.

Exact exhaustive search:

$O(2^S \times S \times U)$

SPACE COMPLEXITY

$O(S \times U)$

RESULT

The greedy Set Cover algorithm successfully covers the entire universe using 3 sets. For this particular input, the greedy solution is also an optimal solution.

5. Heuristic Algorithm for Bin Packing

AIM

To implement the First-Fit heuristic algorithm for the Bin Packing problem and evaluate its number of bins and computational complexity.

ALGORITHM

1. Start with zero bins.
2. Take each item one by one.
3. Place the item into the first bin where it fits.
4. If it does not fit into any existing bin, create a new bin.
5. Continue until all items are placed.
6. Count the number of bins used.

SOURCE CODE

```
1 #include <stdio.h>
2 #define N 6
3 int main() {
4     int items[N] = {4, 8, 1, 4, 2, 1};
5     int capacity = 10;
6     int bins[N][N] = {0};
7     int remaining[N];
8     int binCount = 0;
9     for (int i = 0; i < N; i++) {
10         remaining[i] = capacity;
11         for (int i = 0; i < N; i++) {
12             int placed = 0;
13             for (int j = 0; j < binCount; j++) {
14                 if (remaining[j] >= items[i]) {
15                     for (int k = 0; k < N; k++) {
16                         if (bins[j][k] == 0) {
17                             bins[j][k] = items[i];
18                             break;
19                         }
20                     }
21                     remaining[j] -= items[i];
22                     placed = 1;
23                 }
24             }
25             if (!placed) {
26                 binCount++;
27             }
28         }
29     }
30     printf("Number of Bins Used: %d\n", binCount);
31     for (int i = 0; i < binCount; i++) {
32         printf("Bin %d: [", i + 1);
33         int first = 1;
34         for (int j = 0; j < N; j++) {
35             if (bins[i][j] != 0) {
36                 if (!first) {
37                     printf(", ");
38                 }
39                 printf("%d", bins[i][j]);
40                 first = 0;
41             }
42         }
43         printf("]\n");
44     }
45     printf("Time Complexity: O(n^2) for this straightforward First-Fit implementation\n");
46     return 0;
47 }
```

SAMPLE INPUT AND OUTPUT

```
Greedy Set Cover: {3, 4, 5, 6}, {1, 2, 3}, {5, 6, 7}
Optimal Set Cover: {1, 2, 3}, {2, 4}, {5, 6, 7}
Greedy uses 3 sets.
Optimal uses 3 sets.
```

```
=== Code Execution Successful ===
```

TIME COMPLEXITY

For the straightforward First-Fit implementation:

$O(n^2)$

With more sophisticated data structures, the practical performance can be improved.

SPACE COMPLEXITY

$O(n)$

RESULT

The First-Fit heuristic successfully packs all items into 2 bins, each with capacity 10. It provides a fast approximate solution to the NP-hard Bin Packing problem.

6. Approximation Algorithm for Maximum Cut

AIM

To implement a greedy approximation algorithm for the Maximum Cut problem and compare its result with the optimal solution obtained through exhaustive search.

ALGORITHM

Greedy Approach

1. Divide the vertices into two sets.
2. Initially place vertices into a partition.
3. For every vertex, determine whether moving it to the opposite partition increases the cut weight.
4. Perform the move if it improves the cut.
5. Repeat until no improvement is possible.

Exact Approach

1. Enumerate all possible partitions of the vertices.
2. Calculate the total weight of edges crossing the two partitions.
3. Store the partition having maximum cut weight.
4. Compare the greedy result with the exact result.

SOURCE CODE

```
1 #include <stdio.h>
2 #define V 4
3 #define E 5
4 typedef struct {
5     int u, v, weight;
6 } Edge;
7 Edge edges[E] = {
8     {1, 2, 2},
9     {1, 3, 1},
10    {2, 3, 3},
11    {2, 4, 4},
12    {3, 4, 2}
13 };
14 int cutWeight(int side[]) {
15     int weight = 0;
16     for (int i = 0; i < E; i++) {
17         if (side[edges[i].u - 1] != side[edges[i].v - 1])
18             weight += edges[i].weight;
19     }
20     return weight;
21 }
22
23 int main() {
24     int side[V] = {0, 0, 1, 1};
25     int improved = 1;
26     while (improved) {
27         improved = 0;
28         for (int i = 0; i < V; i++) {
29             int current = cutWeight(side);
30             side[i] = 1 - side[i];
31             int newWeight = cutWeight(side);
32             if (newWeight > current) {
33                 improved = 1;
34             } else {
35                 side[i] = 1 - side[i];
36             }
37         }
38     }
39     int greedyWeight = cutWeight(side);
40     printf("Greedy Maximum Cut Weight: %d\n", greedyWeight);
41     printf("Partition 1: {");
42     int first = 1;
43     for (int i = 0; i < V; i++) {
44         if (side[i] == 0) {
```

```

23 int main() {
54     for (int i = 0; i < V; i++) {
55         if (side[i] == 1) {
56             printf("%d", i + 1);
57             first = 0;
58         }
59     }
60 }
61
62 printf("\n");
63 int bestWeight = -1;
64 int bestSide[V];
65 for (int mask = 0; mask < (1 << V); mask++) {
66     int temp[V];
67     for (int i = 0; i < V; i++)
68         temp[i] = (mask >> i) & 1;
69     int weight = cutWeight(temp);
70     if (weight > bestWeight) {
71         bestWeight = weight;
72         for (int i = 0; i < V; i++)
73             bestSide[i] = temp[i];
74     }
75 }
76 printf("Optimal Maximum Cut Weight: %d\n", bestWeight);
77 printf("Optimal Partition 1: {");

```

```

23 int main() {
79     for (int i = 0; i < V; i++) {
80         if (bestSide[i] == 0) {
81             printf("%d", i + 1);
82             first = 0;
83         }
84     }
85 }
86
87 printf("\n");
88 printf("Optimal Partition 2: {");
89 first = 1;
90 for (int i = 0; i < V; i++) {
91     if (bestSide[i] == 1) {
92         if (!first)
93             printf(", ");
94         printf("%d", i + 1);
95         first = 0;
96     }
97 }
98 printf("\n");
99 printf("Performance = %.2f%%\n",
100        ((double)greedyWeight / bestWeight) * 100);
101 return 0;
102 }

```

SAMPLE INPUT AND OUTPUT

```

Output

Greedy Maximum Cut Weight: 9
Partition 1: {2}
Partition 2: {1, 3, 4}
Optimal Maximum Cut Weight: 9
Optimal Partition 1: {1, 3, 4}
Optimal Partition 2: {2}
Performance = 100.00%

=== Code Execution Successful ===

```

TIME COMPLEXITY

Greedy local-search implementation:

$O(V \times E)$ per pass; potentially multiple passes.

For the small graph, this is easily manageable.

Exhaustive search:

$O(2^V \times E)$

SPACE COMPLEXITY

$O(V + E)$

RESULT

The greedy Maximum Cut algorithm successfully obtains a high-quality cut, and exhaustive search confirms that the optimal cut weight for the given graph is 9.